Digital Signal Processing Exercise Sheet 5

Nyquist

Script: Chapter 5 (Sampling Theorem, Aliasing, Reconstruction) **with solutions**

Learning goals

- Predict alias frequencies with the folding rule and the folding diagram; determine minimum sampling rates, including band-pass (under-)sampling.
- Observe aliasing of the knock resonances on real (simulated) microcontrollers and verify the predictions quantitatively.
- Compare zero-order hold, linear and sinc reconstruction.
- Design an anti-aliasing filter: trade-off between filter order, sampling rate and passband loss.

Setup: JSLinux

From this sheet on, the lab exercises also run in JSLinux, a Linux system in the browser. Sheets 0–4 ran entirely in Wokwi; if you have not done the JSLinux bonus exercises of Sheets 0–2, set it up now (about 10 min):

1. Open `https://bellard.org/jslinux`, start the *x86 Alpine Linux* (console) virtual machine and check the compiler with `gcc -version` (or `tcc -v`).
2. Upload the course headers `engine_signals.h`, `asciiplot.h`, `csvio.h` (`code/common/`) and the files of the exercise (e.g. `code/jslinux/sheet05/`) with the upload button below the terminal; check with `ls` that they arrived. Headers and program must be in the same directory.
3. Compile and run, e.g.

```
gcc -O2 -o recon recon.c -lm
./recon
```

4. The VM forgets all files when the page is reloaded: keep your sources on your own computer and upload them again. Text from Wokwi (Serial Monitor) gets into the VM with `cat > data.csv`, paste, `Ctrl-D`; `export_file data.csv` downloads a file from the VM to your computer.

Background

Sampling with $f_s = 1/T_s$ makes all frequencies $f + k f_s$, $k \in \mathbb{Z}$, indistinguishable. A real tone of frequency f therefore appears at the *alias frequency*

$$f_a = |\, f - k f_s|, \qquad k = \mathrm{round}(f/f_s), \qquad 0 \le f_a \le f_s/2.$$

A band-limited signal ($X(f) = 0$ for $|f| \ge B$) is uniquely determined by its samples if $f_s > 2B$ and can be reconstructed by sinc interpolation $x(t) = \sum_n x[n]\,\mathrm{sinc}\big((t - nT_s)/T_s\big)$, $\mathrm{sinc}(u) = \sin(\pi u)/(\pi u)$. Everything above $f_s/2$ has to be removed *before* sampling by an analog anti-aliasing (AA) filter.

In our virtual engine the knock sensor has resonances at 6.5 kHz and 10.5 kHz ($\tau = 0.8$ ms) and valve impacts at 8 kHz ($\tau = 0.4$ ms). The `engine-sim` chip updates its analog outputs every 20 µs (attribute `fs_khz = 50`) and holds the value in between: the chip itself is a sampled system with zero-order hold.

Exercise 5.1 Aliasing by hand

PAPER

a) Compute the alias frequencies of the components 6.5 kHz, 8 kHz, 10.5 kHz and of a (hypothetical) third knock mode at 15 kHz for $f_s = 5$, 8.33, 16.67, 25 and 50 kHz. (8.33 kHz is close to the maximum rate of the Uno ADC.)

b) Draw the folding diagram (f_a over f, 0 kHz to 16 kHz) for $f_s = 8.33$ kHz and mark the four components. Which two components can no longer be separated after sampling?

c) What is the minimum sampling rate for a knock signal with components up to 15 kHz? A 4th-order Butterworth AA filter with $f_c = 15$ kHz ($|H| = 1/\sqrt{1 + (f/f_c)^8}$) is used. Which f_s is needed so that every frequency that folds into 0 kHz to 15 kHz is attenuated by at least 40 dB?

d) Band-pass sampling: an analog band-pass passes only the first knock mode, 6 kHz to 7 kHz. Which sampling rates are allowed ($2f_H/k \le f_s \le 2f_L/(k-1)$, $k = 1, 2, \dots$)? Show that the Uno at 5 kHz could legally capture this band. Where does the band appear, and is it spectrally inverted?

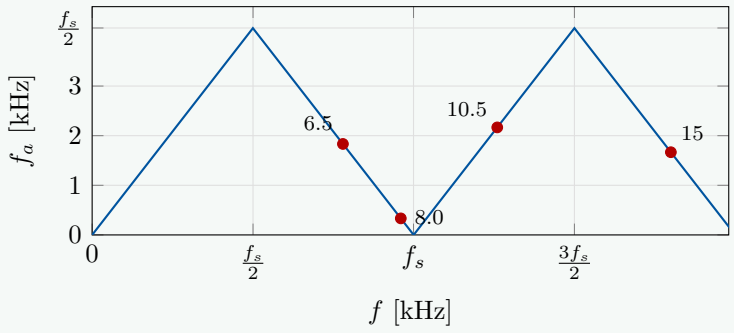
Solution

a) Alias frequencies in kHz (k in brackets):

f	$f_s = 5$	8.33	16.67	25	50
6.5 (knock 1)	1.5 (1)	1.833 (1)	6.5 (0)	6.5	6.5
8.0 (valve)	2.0 (2)	0.333 (1)	8.0 (0)	8.0	8.0
10.5 (knock 2)	0.5 (2)	2.167 (1)	6.167 (1)	10.5	10.5
15.0 (knock 3)	0 (3)	1.667 (2)	1.667 (1)	10.0 (1)	15.0

Notable cases: at 5 kHz the 15 kHz mode becomes DC; at 16.67 kHz the second knock mode lands at 6.17 kHz, right next to the first mode – a 6.5 kHz band-pass would see it; at 25 kHz a 15 kHz mode would mimic 10 kHz.

b) Folding diagram for $f_s = 8.33$ kHz (zig-zag with corners at multiples of $f_s/2$):



The two knock modes end up at 1.83 kHz and 2.17 kHz: only 333 Hz apart, which is less than the frequency resolution of a knock burst of a few ms (Sheet 8). Mode 3 (1.67 kHz) falls into the same region; the valve impact becomes an almost-DC transient.

c) Nyquist: $f_s > 30$ kHz. With an AA filter, a frequency f folds into $[0, 15]$ kHz if $f \ge f_s - 15$ kHz; the weakest attenuation occurs at $f = f_s - 15$. 40 dB requires $(f/f_c)^8 \ge 10^4$, i.e. $f \ge f_c \cdot 10^{0.5} = 47.4$ kHz, so $f_s \ge 62.4$ kHz (in practice 64 kHz). The theoretical minimum of 30 kHz would need an ideal brick-wall filter.

d) $f_L = 6$, $f_H = 7$, $B = 1$ kHz, $k \le \lfloor f_H/B \rfloor = 7$:

k	1	2	3	4	5	6	7
f_s [kHz]	≥ 14	7–12	4.67–6	3.5–4	2.8–3	2.33–2.4	2

$f_s = 5$ kHz lies in the $k = 3$ zone. The band lies in the third Nyquist zone $[5, 7.5]$ kHz (odd zone, not inverted) and appears at $f - f_s$, i.e. at 1 kHz to 2 kHz. Condition: the analog band-pass must suppress the valve (8 kHz→2 kHz) and mode 2 (10.5 kHz→0.5 kHz), which fold into the same 0 kHz to 2.5 kHz range. Band-pass sampling replaces a high f_s by a demanding analog filter.

Exercise 5.2 Aliasing made visible

WOKWI

Two projects record the `KNOCK` output of the engine-sim chip (knock probability 100 %, 3000 rpm) together with the ground-truth outputs `KNK` (HIGH during a knock burst) and `TDC`, and print blocks of CSV lines `n,adc,knk,tdc`: `code/wokwi/sheet05_alias_uno/` (Uno, Timer1 ISR, 512 samples per block) and `code/wokwi/sheet05_alias_esp32/` (ESP32, deadline loop, 2500 samples per block). The JSLinux program `code/jslinux/sheet05/alias_scan.c` cuts out every knock burst (from the rising KNK edge, 4τ) and every valve burst (TDC $+90°$, 5τ), averages the spectra $|X(f)|^2$ of the segments and lists the peaks.

a) Uno: the sampling period is `TS_US = DIV`×20 µs (default `DIV = 10`, 5 kHz). Compute `OCR1A` (prescaler 8) and complete the ISR (store KNK in bit 14, TDC in bit 15). Why does the sketch disable the Timer0 interrupt during acquisition?

b) Copy at least three blocks from the Serial Monitor into `uno.csv`, implement `alias_freq()` and the start index of the valve burst in `alias_scan.c` and run `./alias_scan uno.csv 5000 10`. Compare the spectral peaks with Exercise 5.1. Look at one knock burst in the CSV: how many oscillation periods do you see, and of which frequency? Repeat with `DIV = 6` (8.33 kHz). Check with the logic analyzer that the sample marker (D0) is still regular.

c) ESP32: complete the deadline loop (`TS_US = 40`, 25 kHz) and verify that no sample is late. Analyse with `./alias_scan esp.csv 25000 12`. Then set `TS_US = 60` (16.67 kHz). Why is this particularly dangerous for a knock detector that integrates the energy in a band around 6.5 kHz?

d) Why must the sampling period be a multiple of 20 µs? Consider the Uno at exactly 8 kHz (125 µs): determine the sequence of time offsets between the ADC instants and the last chip update, its effect on the SNR (Exercise 4.1 e) and a value of the chip attribute `fs_khz` in `diagram.json` that fixes the problem.

e) `./alias_scan sim 5000 10` simulates the chip and the ADC (“virtual Wokwi”, also writes `alias_sim.csv`). Compare its peaks with your Wokwi measurement.

The UART transmits about 11.5 characters per ms: printing a CSV line per sample is impossible at kHz rates, therefore samples are buffered in RAM (Uno: 512 samples = 1 KB of its 2 KB). Wokwi is not cycle-accurate for the ESP32 – if `late` is not 0, increase `TS_US` to the next multiple of 20.

Solution

a) `OCR1A` $= 2 \cdot$ `TS_US` $- 1 = 399$ (2 MHz timer clock).

```
ISR(TIMER1_COMPA_vect)
{
    PINB = _BV(PB0);                          /* toggle pin 8: sample marker */
    uint16_t v = analogRead(A0);
    if (PIND & _BV(PD3)) v |= 0x4000;         /* KNK (pin 3)                 */
    if (tdc_seen) { v |= 0x8000; tdc_seen = 0; }
    buf[nbuf++] = v;
    if (nbuf >= NBUF) { TIMSK1 = 0; done = true; }
}

void timer1_start()
{
    noInterrupts();
    TCCR1A = 0;
    TCCR1B = _BV(WGM12) | _BV(CS11);          /* CTC, clk/8 = 2 MHz          */
    OCR1A  = 2 * TS_US - 1;                   /* 2 ticks per microsecond     */
    TCNT1  = 0;
    TIFR1  = _BV(OCF1A);
    TIMSK1 = _BV(OCIE1A);
    TIMSK0 = 0;           /* stop the millis() interrupt: no extra jitter  */
```

```
    interrupts();
}
```

The Timer0 overflow ISR (`millis()`, every 1.024 ms, a few µs long) can delay the Timer1 ISR, i.e. cause sampling jitter (Sheet 4); the Timer1 ISR itself starts with a constant latency.
b) Results of the virtual Wokwi (`sim`, 400 ms, 39 knock and 38 valve bursts); your Wokwi recording should agree within the resolution:

	predicted (5.1)		measured peaks	
f_s [kHz]	knock modes	valve	knock bursts	valve bursts
5 (Uno)	1.5 / 0.5	2.0	1.51, 0.255 (34 %)	1.78, 0.98 (83 %)
8.33 (Uno)	1.833 / 2.167	0.333	1.85	0.367
25 (ESP32)	6.5 / 10.5	8.0	6.50, 10.525 (24 %)	8.025
16.67 (ESP32)	6.5 / 6.167	8.0	6.483	8.183

(all values in kHz; percentages: power relative to the largest peak). At 5 kHz the 6.5 kHz knock appears at 1.5 kHz: a burst of $4\tau = 3.2$ ms (16 samples) shows about 5 periods of 1.5 kHz, i.e. the "wrong" frequency is clearly visible in the time signal. The second peak (alias of 10.5 kHz at 0.5 kHz) is shifted to 255 Hz because the segment of only 3.2 ms resolves no better than $\approx 1/3.2$ ms ≈ 300 Hz and the combustion vibration adds low-frequency content. The valve burst segments (2 ms, 10 samples) have ≈ 500 Hz resolution; the peak at 1.78 kHz instead of 2.0 kHz and the side peak result from the overlap with the mirrored component at $f_s - 2$ kHz $= 3$ kHz. At 8.33 kHz the two knock aliases (333 Hz apart) merge into one peak at 1.85 kHz, and the 8 kHz valve burst looks like a slow 0.33 kHz transient. The Uno at 8.33 kHz works only because the CPU does nothing but sampling during the block ($\approx$ 112 µs of every 120 µs); D0 must show regular 120 µs steps.
c) Deadline loop:

```
    t_next = micros() + TS_US;
    for (i = 0; i < NBUF; i++) {
        while ((int32_t)(micros() - t_next) < 0) { }
        if ((int32_t)(micros() - t_next) > (int32_t)(TS_US / 2)) late++;
        t_next += TS_US;
        digitalWrite(PIN_MARK, mark ^= 1);
        uint16_t v = analogRead(PIN_KNOCK);
        if (digitalRead(PIN_KNK)) v |= 0x4000;
        if (tdc_seen) { v |= 0x8000; tdc_seen = 0; }
        buf[i] = v;
    }
```

At 25 kHz all components lie below $f_s/2 = 12.5$ kHz and appear at their true frequencies (6500 Hz, 10 525 Hz with power ratio $0.24 \approx 0.5^2$ as in the model, valve 8025 Hz). At 16.67 kHz mode 2 aliases to 6.17 kHz and merges with mode 1 into a single peak at 6.48 kHz. A band-pass detector at 6.5 kHz integrates the energy of both modes, and every disturbance between 10 kHz and 11 kHz (e.g. ignition or injector noise) would be counted as knock – aliasing produces false knock detections that no digital filter can remove afterwards.
d) Sampling a held (staircase) signal with a period that is a multiple of 20 µs is equivalent to exact decimation of the chip's 50 kHz sequence. With 125 µs the offset between ADC instant and last chip update cycles through $125n \bmod 20 = 0, 5, 10, 15, 0, \ldots$ µs. The mean (7.5 µs) is a harmless delay, the varying part has $\sigma = 5.6$ µs, i.e. SNR $= -20\log_{10}(2\pi \cdot 6500 \cdot 5.6 \cdot 10^{-6}) \approx 12.8$ dB for mode 1 (8.7 dB for mode 2), concentrated in spurious lines at $\pm f_s/4$ around each component because the offset is periodic. Fix: `"fs_khz": "40"` (update every 25 µs, $125 = 5 \cdot 25$; the chip's own Nyquist limit 20 kHz is still above 10.5 kHz) or `"200"` (5 µs, more simulation load).
e) The simulation produces the peaks of the table in b); the Wokwi recordings differ only by the random knock amplitudes/phases (the chip uses its own random sequence) and the number of bursts.

Exercise 5.3 Reconstruction and anti-aliasing JSLINUX

Reconstruction. `code/jslinux/sheet05/recon.c` represents a "continuous" sine on a grid of $M = 40$ points per sampling interval ($f_s = 25\,\text{kHz}$, 600 samples) and computes the normalised error $\text{NMSE} = \sum e^2 / \sum x^2$ of a reconstruction on this grid.

a) Implement zero-order hold, linear interpolation and sinc interpolation with $\pm K = 16$ terms, once plain (truncated) and once with a Hann taper $w(u) = \frac{1}{2} + \frac{1}{2}\cos(\pi u/(K+1))$. Show analytically that the ZOH of a sine has $\text{NMSE} = 2(1 - \sin(\omega T_s)/(\omega T_s))$ and compare. Discuss the table (method vs. f/f_s).

b) A 15 kHz tone is sampled at 25 kHz and sinc-reconstructed. What comes out? Predict frequency *and* phase.

Anti-aliasing. `code/jslinux/sheet05/antialias.c` simulates the analog world on a 1 MHz grid; the AA filter (cut-off $f_c = 12\,\text{kHz}$) is simulated on this grid, the ADC takes every ($1\,\text{MHz}/f_s$)-th value. Test signals: knock tones 6.5 kHz/10.5 kHz; a disturbance at $f_d = f_s - 6.5\,\text{kHz}$ that aliases exactly onto the knock frequency; white noise (bandwidth 500 kHz).

c) Implement the RC low-pass (exact update for an input held constant over Δt), the 4th-order Butterworth as a cascade of two 2nd-order sections ($Q_k = 1/(2\sin\frac{(2k-1)\pi}{2n})$, given biquad simulator) and the amplitude measurement at a given frequency in the sampled signal. Run the program for $f_s = 25$ and 50 kHz and filter orders 0, 1, 2, 4. Compare the alias amplitudes with $|H(f_d)|$.

d) Which Butterworth order would be required for 40 dB alias suppression in the band 0 kHz to 10.5 kHz with $f_c = 12\,\text{kHz}$ at $f_s = 25\,\text{kHz}$ and at 50 kHz? Formulate a design recommendation for the knock ADC.

Solution

a) Implementations:

```
static void rec_zoh(void)
{
    int i;
    for (i = 0; i < NF; i++) xr[i] = xs[i / M];
}

static void rec_lin(void)
{
    int i;
    for (i = 0; i < NF; i++) {
        int n = i / M;
        double u = (double)(i % M) / M;
        xr[i] = n + 1 < NSMP ? (1.0 - u) * xs[n] + u * xs[n + 1] : xs[n];
    }
}

static void rec_sinc(int windowed)
{
    int i, k;
    for (i = 0; i < NF; i++) {
        double tn = (double)i / M;              /* time in sample periods */
        int n = i / M;
        double s = 0.0;
        for (k = n - K; k <= n + K + 1; k++) {
            double u = tn - k, w = 1.0;
            if (k < 0 || k >= NSMP) continue;
            if (windowed) {
                if (fabs(u) >= K + 1) continue;
                w = 0.5 + 0.5 * cos(M_PI * u / (K + 1));   /* Hann taper */
```

```
            }
            s += xs[k] * sinc(u) * w;
        }
        xr[i] = s;
    }
}
```

ZOH error of $x = \sin(\omega t + \varphi)$ at $t = nT_s + u$: $e = \sin(\alpha + \omega u) - \sin\alpha$; averaged over the phase α: $\overline{e^2} = \frac{1}{2}|e^{j\omega u} - 1|^2 = 1 - \cos\omega u$; averaged over $u \in [0, T_s)$: $1 - \sin(\omega T_s)/(\omega T_s)$; division by the signal power $1/2$ gives the formula. Program output (NMSE in dB):

f [Hz]	f/f_s	ZOH	ZOH theory	linear	sinc	Hann-sinc
500	0.02	−22.98	−22.79	−56.83	−42.94	−97.22
1000	0.04	−16.94	−16.78	−44.79	−44.98	−99.46
2500	0.10	−9.06	−8.89	−28.94	−47.59	−101.24
5000	0.20	−3.28	−3.13	−17.12	−37.44	−84.43
8000	0.32	0.27	0.41	−9.43	−32.66	−70.23
10500	0.42	2.02	2.13	−5.28	−30.83	−57.78
12000	0.48	2.76	2.83	−3.38	−27.58	−11.36

The ZOH agrees with the theory within 0.2 dB (the fine grid contains $u = 0$ with zero error, hence slightly smaller values). The ZOH is a poor reconstruction: even at $f = f_s/25$ the error is only 17 dB below the signal – mainly due to the delay $T_s/2$ and the staircase; above $f \approx f_s/3.5$ the error is larger than the signal. Linear interpolation gains about 12 dB per octave (error $\propto (\omega T_s)^4$) but is useless near $f_s/2$. The truncated sinc has a frequency-independent floor of about $-30 \ldots -48$ dB caused by cutting the slowly decaying $(1/u)$ kernel; at low frequencies it is even worse than linear interpolation. The Hann taper removes this floor (-97 to -101 dB), but its transition band costs accuracy close to $f_s/2$ (-11 dB at $0.48 f_s$): the closer a signal gets to Nyquist, the longer the interpolation kernel has to be.

b) $\sin(2\pi\, 15000\, nT_s + 0.7) = \sin(2\pi(15000 - 25000)nT_s + 0.7) = \sin(-2\pi\, 10000\, nT_s + 0.7)$: the samples are those of a 10 kHz sine with inverted phase, $-\sin(2\pi\, 10\,\text{kHz}\, t - 0.7)$. The program finds a zero-crossing frequency of 10 000 Hz and NMSE = 3.00 dB against the 15 kHz original, but −59.7 dB against this alias. The reconstruction is perfect – of the wrong signal.

c) Key code:

```
static void rc_filter(double *y, const double *x, int n, double fc, double dt)
{
    double a = exp(-2.0 * M_PI * fc * dt), s = 0.0;
    int i;
    for (i = 0; i < n; i++) {
        s = a * s + (1.0 - a) * x[i];
        y[i] = s;
    }
}
```

```
    case 4: /* Q_k = 1 / (2 sin((2k-1) pi / (2n))), k = 1, 2           */
        biquad_filter(tmp, x, n, FC, 1.0 / (2.0 * sin(M_PI / 8.0)), dt);
        biquad_filter(y, tmp, n, FC, 1.0 / (2.0 * sin(3.0 * M_PI / 8.0)), dt);
        break;
```

```
    for (n = 0; n < N; n++) {
        double v = y[(n0 + n) * D], w = 2.0 * M_PI * f * n / fs;
        re += v * cos(w);
        im -= v * sin(w);
    }
    return 2.0 * sqrt(re * re + im * im) / N;
```

Results (f_c = 12 kHz; "noise" = noise PSD in 5 kHz to 11 kHz after sampling relative to the unaliased analog noise PSD):

f_s	order	gain 6.5k	gain 10.5k	alias of f_d	$\lvert H(f_d)\rvert$	noise
25 kHz	0	0.00	0.00	0.0 dB	0.0 dB	+15.7 dB
($f_d = 18.5$ kHz)	1	−1.12	−2.47	−5.3 dB	−5.3 dB	+2.0 dB
	2	−0.36	−2.01	−8.2 dB	−8.2 dB	0.0 dB
	4	−0.03	−1.29	−15.2 dB	−15.2 dB	0.0 dB
50 kHz	0	0.00	0.00	0.0 dB	0.0 dB	+12.7 dB
($f_d = 43.5$ kHz)	1	−1.12	−2.47	−11.5 dB	−11.5 dB	−0.3 dB
	2	−0.36	−2.01	−22.4 dB	−22.4 dB	−1.0 dB
	4	−0.03	−1.29	−44.8 dB	−44.7 dB	−0.2 dB

(gains in dB.) The sampled alias amplitude equals $|H(f_d)|$ exactly: the filter acts before sampling, the sampler only moves the attenuated component to 6.5 kHz. Without filter, all noise of the 500 kHz band folds into $[0, f_s/2]$: the in-band noise rises by $10\log_{10}(500/12.5) = 16$ dB (measured 15.7 dB) at 25 kHz and by 13 dB at 50 kHz. Already a first-order filter removes most of this broadband penalty, but at $f_s = 25$ kHz even the 4th-order filter suppresses the disturbance by only 15 dB while it costs 1.3 dB at 10.5 kHz.

d) The first frequency that folds into 0 kHz to 10.5 kHz is $f_s - 10.5$ kHz. Requirement $(f/f_c)^{2n} \geq 10^4$: $f_s = 25$ kHz: $f/f_c = 14.5/12 = 1.21$, $n \geq 2/\log_{10} 1.21 = 24.3$, i.e. order 25 – impossible. $f_s = 50$ kHz: $f/f_c = 39.5/12 = 3.29$, $n \geq 2/\log_{10} 3.29 = 3.9$, i.e. order 4 (the table confirms 44.8 dB at 43.5 kHz). Recommendation: oversample (e.g. 50 kHz to 100 kHz), use a simple 2nd–4th-order analog AA filter (Sallen-Key), and reduce the rate digitally afterwards (digital low-pass + decimation, Sheet 10), where steep filters are cheap and exactly reproducible.

Deliverables

5.1: alias table, folding diagram, AA and band-pass sampling calculation. 5.2: completed sketches, CSV files of the Uno (two rates) and the ESP32 (two rates), output of `alias_scan` with comparison to 5.1, answers to c) and d). 5.3: completed programs, both tables with discussion, filter-order recommendation.